

PART7. Docker Volume

쉬운 도커

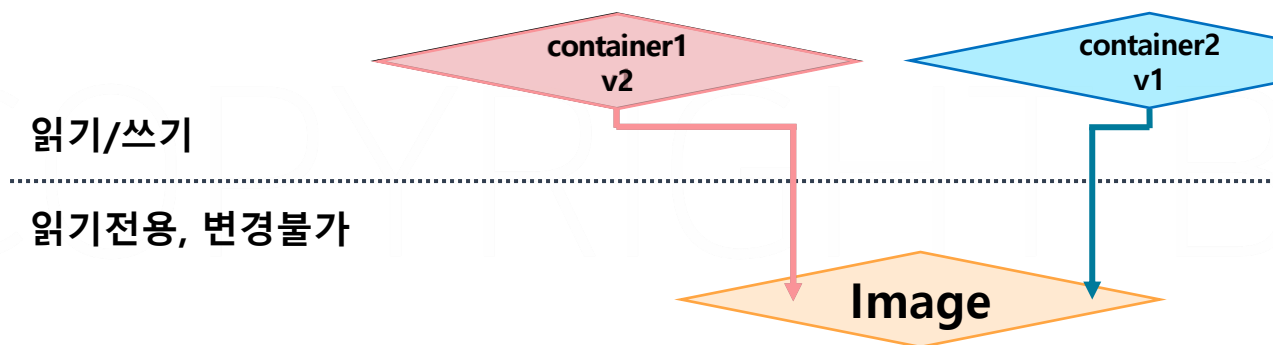
PART7. Docker Volume

PART7. 도커 볼륨

(Docker Volume)

컨테이너의 상태

- 컨테이너는 **상태 없음(Stateless)** 입니다. 컨테이너가 실행 된 후 발생하는 모든 변경 사항은 컨테이너 레이어에만 있으며 컨테이너가 종료되면 변경 사항이 모두 사라집니다.
- 컨테이너는 Stateless하기 때문에 쉽게 개수를 증가시킬 수 있으며 다른 환경에서도 빠르게 배포할 수 있습니다.



1. Container 1, 2 생성

: Container 1, 2 의 읽기/쓰기 레이어 생성

2. Container 2 삭제

: Container 2 의 읽기/쓰기 레이어 삭제

3. Container 1 업데이트

: Container 1 의 읽기/쓰기 레이어 삭제 후
새로운 Container 1 의 읽기/쓰기 레이어 생성

- 컨테이너는 **상태 없음(Stateless)** 입니다. 컨테이너가 실행 된 후 발생하는 모든 변경 사항은 컨테이너 레이어에만 있으며 컨테이너가 종료되면 변경 사항이 모두 사라집니다.
- 컨테이너는 Stateless하기 때문에 쉽게 개수를 증가시킬 수 있으며 다른 환경에서도 빠르게 배포할 수 있습니다.
- 소프트웨어의 버전 등 컨테이너의 상태 변경이 필요한 경우 **새로운 버전의 이미지를 만들어서 배포**합니다.

EX) Nginx 버전을 업데이트 할 경우

기존 기기에 접속해
변경사항 적용

Nginx 1.21.1 -> 1.23.2 업그레이드

Virtual Machine

Hypervisor

새로운 버전의 이미지를
제작하여 재배포

Nginx 1.21.1
Image

Nginx 1.23.2
Image

Docker

- 컨테이너는 **상태 없음(Stateless)** 입니다. 컨테이너가 실행 된 후 발생하는 모든 변경 사항은 컨테이너 레이어에만 있으며 컨테이너가 종료되면 변경 사항이 모두 사라집니다.
- 컨테이너는 Stateless하기 때문에 쉽게 개수를 증가시킬 수 있으며 다른 환경에서도 빠르게 배포할 수 있습니다.
- 소프트웨어의 버전 등 컨테이너의 상태 변경이 필요한 경우 **새로운 버전의 이미지를 만들어서 배포**합니다.
- 컨테이너는 상태가 없기 때문에 여러 대의 컨테이너를 여러 곳에 빠르게 배포할 수 있습니다.



app:v1 app:v1

개발환경



app:v1 app:v1

테스트환경



app:v1 app:v1

운영환경

- 클라우드 네이티브 환경에서는 MSA 아키텍처에 따라 서버의 개수가 매우 많아집니다.

모던 애플리케이션의 요구사항을 충족시키기 위해 서버 관리 방법론이 변화했습니다.

- 전통적인 서버 방법론은 서버 한대를 중요하게 생각하는 Pet 방식입니다. 서버 한 대를 소중하게 케어합니다.

- 컨테이너를 활용한 서버 방법론은 Cattle 방식입니다. 서버를 빠르게 교체할 수 있으며 서버의 상태를 최대한 제거합니다.

기준	Pet 방식	Cattle 방식
방식	전통적, VM 방식	컨테이너 방식
이름	고유한 이름을 가짐	랜덤한 일련번호 생성
문제 발생 시	문제 해결 또는 복구 시도	삭제 후 새로 생성
상태	상태가 내부에 저장	상태 없음, 필요 시 외부 마운트
교체	교체가 어려움	쉽게 교체
적용 사례	Monolithic, OnPremise	MSA, WEBAPP

컨테이너 Stateless 속성 확인

1. 컨테이너 생성

```
docker run -d --name my-nginx nginx
```

2. index.html 파일 작성하여 my-index-file 작성

3. docker cp 명령을 통해 컨테이너 내부로 파일 복사

```
docker cp index.html /usr/share/nginx/html/index.html
```

4. 삭제 및 재실행

```
docker rm -f my-nginx
```

```
docker run -d --name my-nginx nginx
```

4. docker cp 명령을 통해 새롭게 실행한 컨테이너의 index.html 파일 복사

```
docker cp /usr/share/nginx/html/index.html indexfromcontainer.html
```

5. 파일의 내용이 이미지의 초기 index.html인 것을 확인 후 실습 종료

```
docker rm -f my-nginx
```


컨테이너의 이미지는 한번 지정된 후 변경되지 않습니다. (불변성, Immutability)
새로운 설정이나 패치가 필요할 경우 새로운 이미지를 만들어야 합니다.

컨테이너는 언제든지 새로운 컨테이너로 대체할 수 있습니다.

컨테이너는 어떤 호스트에서든 컨테이너를 실행할 수 있습니다.

컨테이너는 동일한 컨테이너를 여러개 쉽게 생성해서 트래픽에 대응할 수 있습니다.

장애가 발생한 경우 새로운 컨테이너를 빠르게 시작할 수 있습니다.

데이터를 영구적으로 저장하기 위해서는 데이터베이스 서버 사용이 필수입니다.
상태가 없기 때문에 **저장 및 공유가 필요한 데이터는 무조건 외부에 저장**해야 합니다.

사용자 세션 정보나 캐시 같은 정보를 **캐시 서버나 쿠키를 통해 관리**합니다.
파일이나 메모리에 저장하지 않아야 합니다.

동일한 요청은 항상 동일한 결과를 제공해야 합니다.
서버마다 다른 응답을 제공하면 안됩니다.

환경 변수나 구성 파일을 통해 **설정을 외부에서 주입**할 수 있어야 합니다.
다양한 환경에서 컨테이너 이미지를 활용할 수 있습니다.

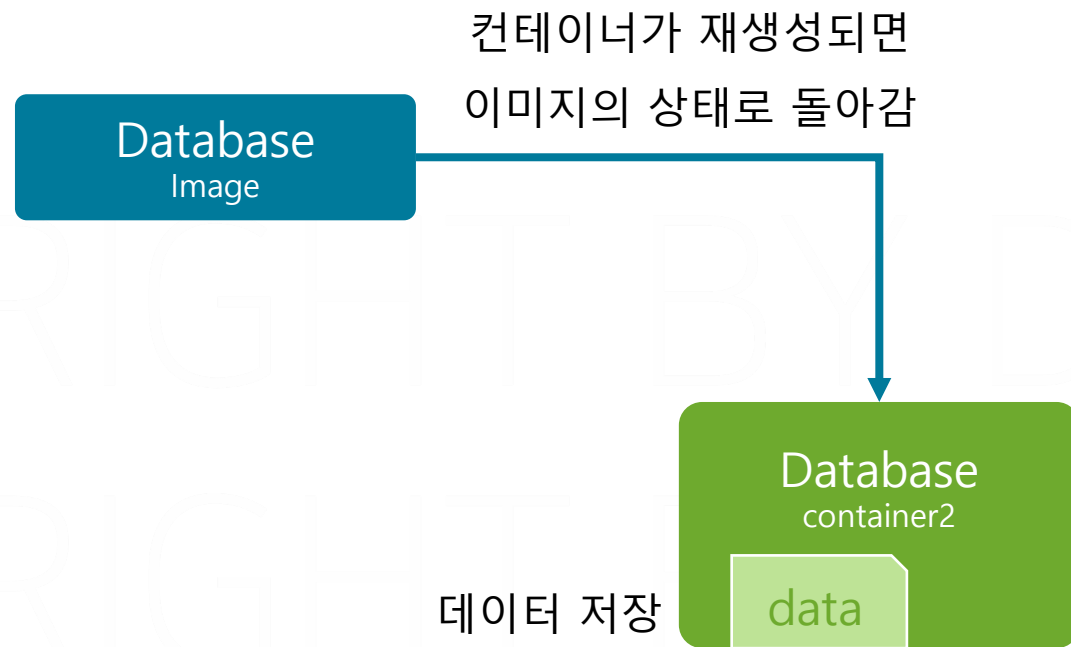
Postgresql Database

```
spring.datasource.driver-class-name=org.postgresql.Driver  
spring.datasource.url=jdbc:postgresql://${DB_URL:localhost}:${DB_PORT:5432}/${DB_NAME:mydb}  
spring.datasource.username=${DB_USERNAME:myuser}  
spring.datasource.password=${DB_PASSWORD:mypassword}
```

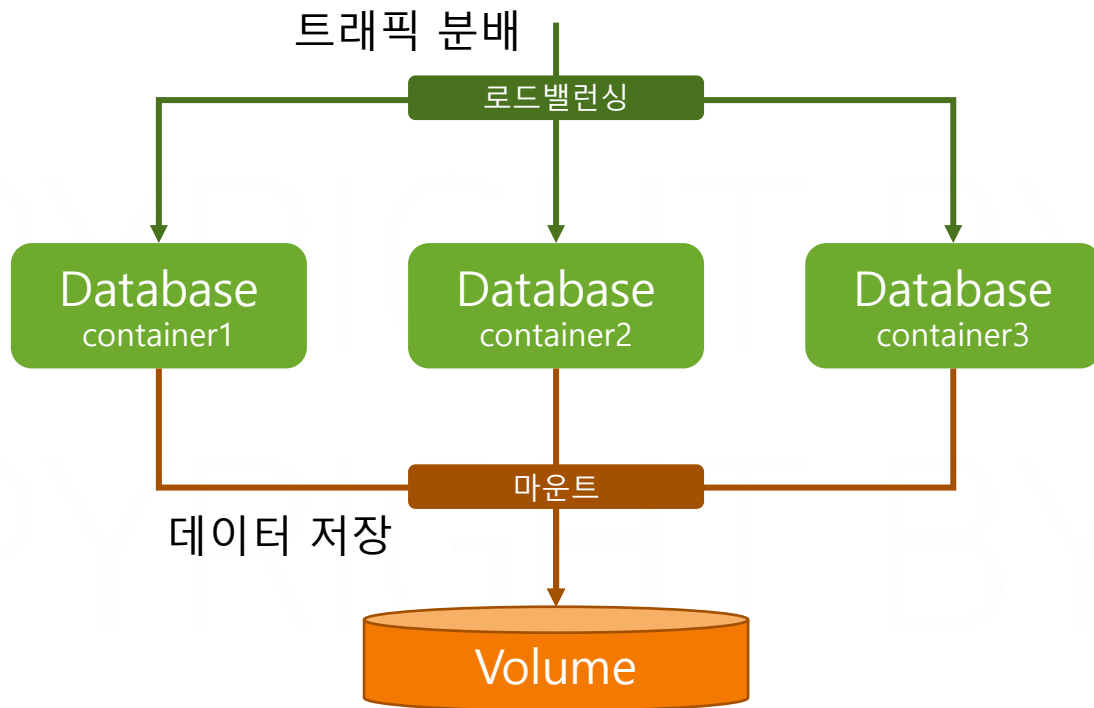
PART7. 스토리지

도커 볼륨

- 컨테이너가 삭제되거나 재생성될 경우 컨테이너레이어가 초기화됩니다.

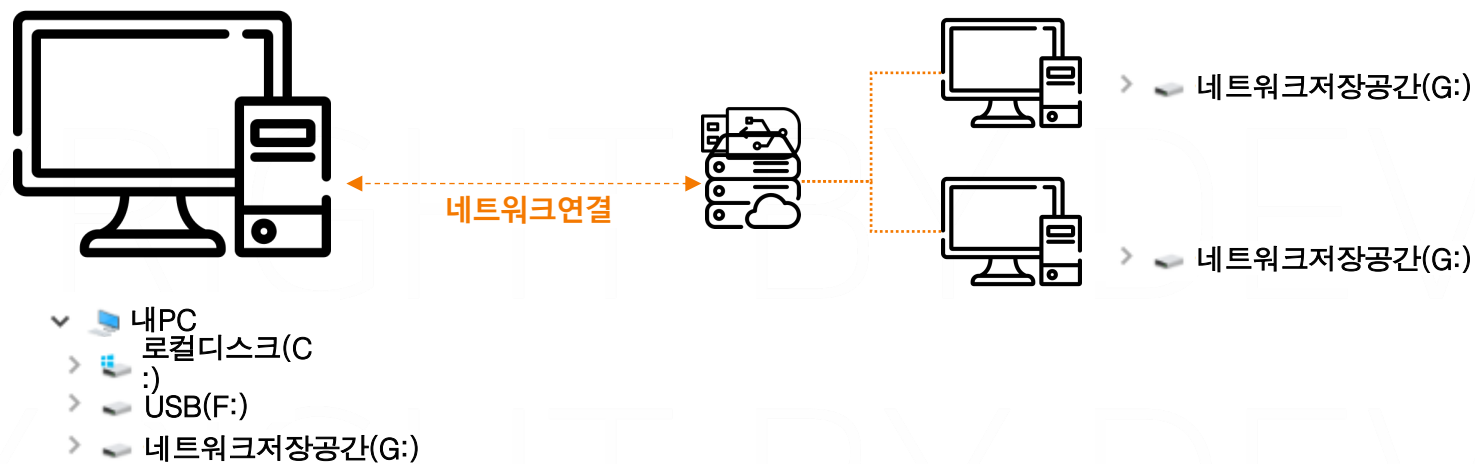


- 컨테이너가 삭제되거나 업그레이드될 경우 컨테이너레이어가 초기화됩니다.
- 컨테이너 환경에서는 같은 서버의 대수가 여러 개 존재합니다.

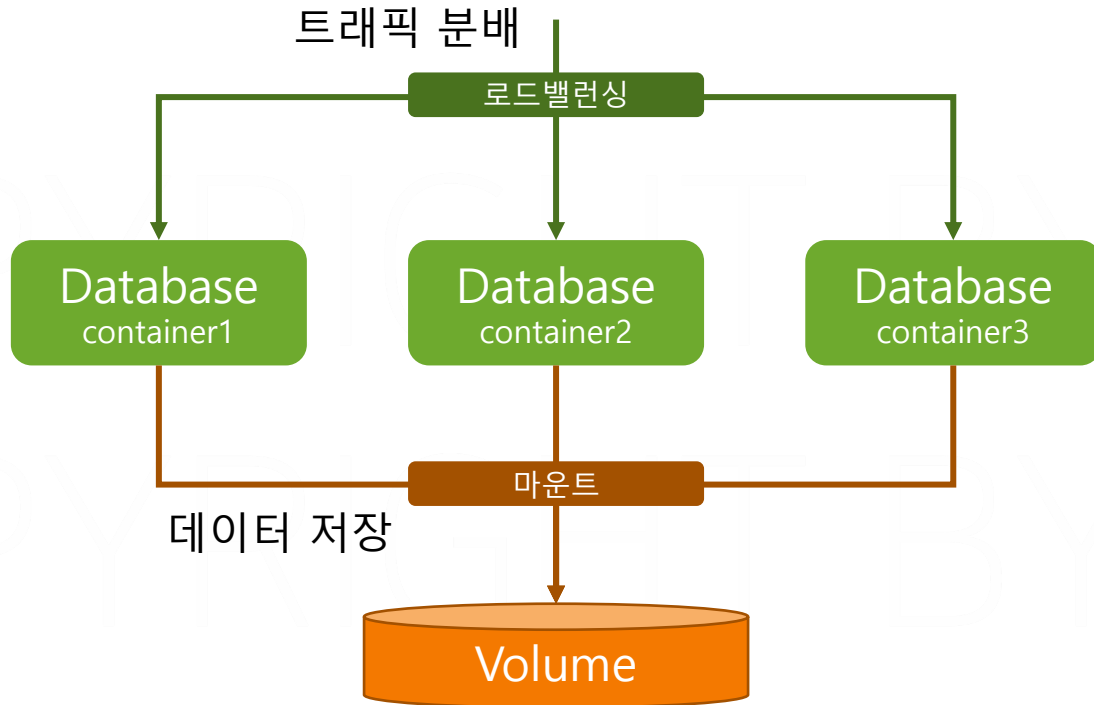


트래픽이 증가할 때마다
컨테이너의 대수 증가

- 마운트를 통해 외부 저장 공간을 특정 경로에 연결할 수 있습니다.
- 외부 저장공간은 물리적으로 연결하거나, 네트워크에 연결할 수 있습니다.



- 컨테이너가 삭제되거나 재생성 될 경우 컨테이너레이어가 초기화됩니다.
- 컨테이너 환경에서는 같은 서버의 대수가 여러 개 존재합니다.
- 컨테이너의 특정 디렉터리에 볼륨을 마운트해서 사용합니다.

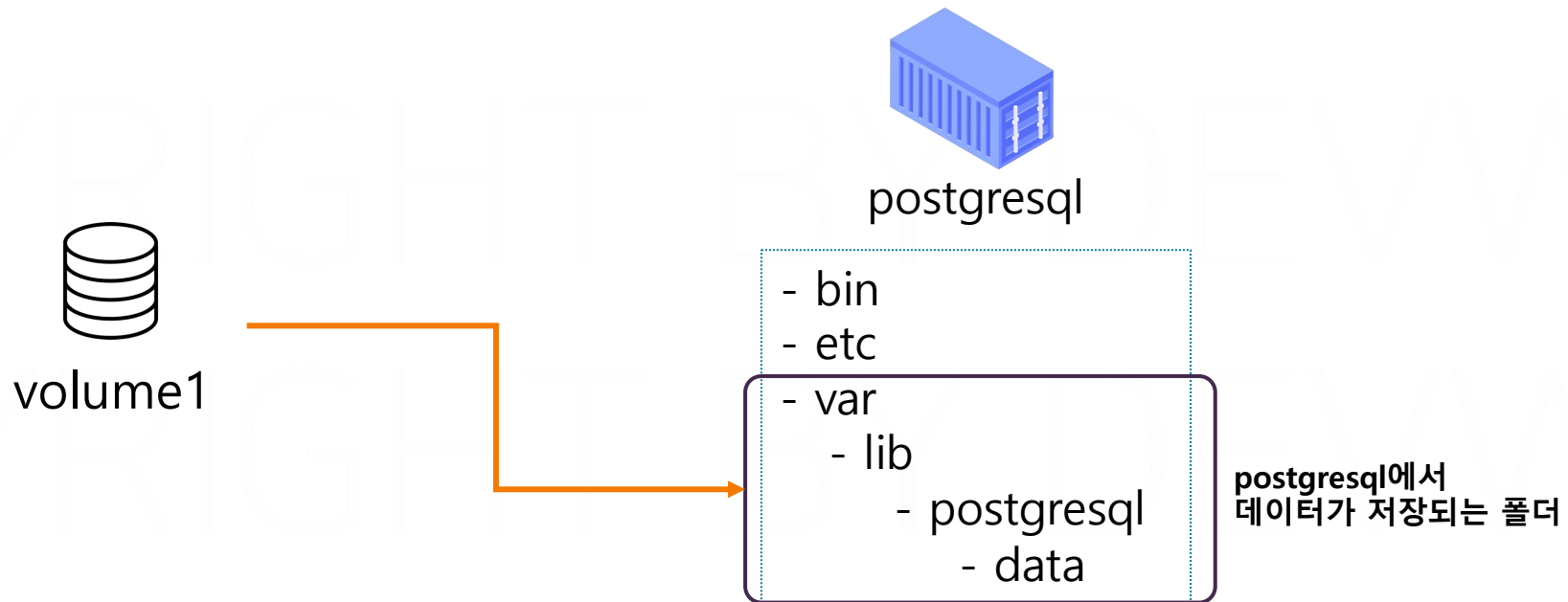


/var/lib/postgresql/data

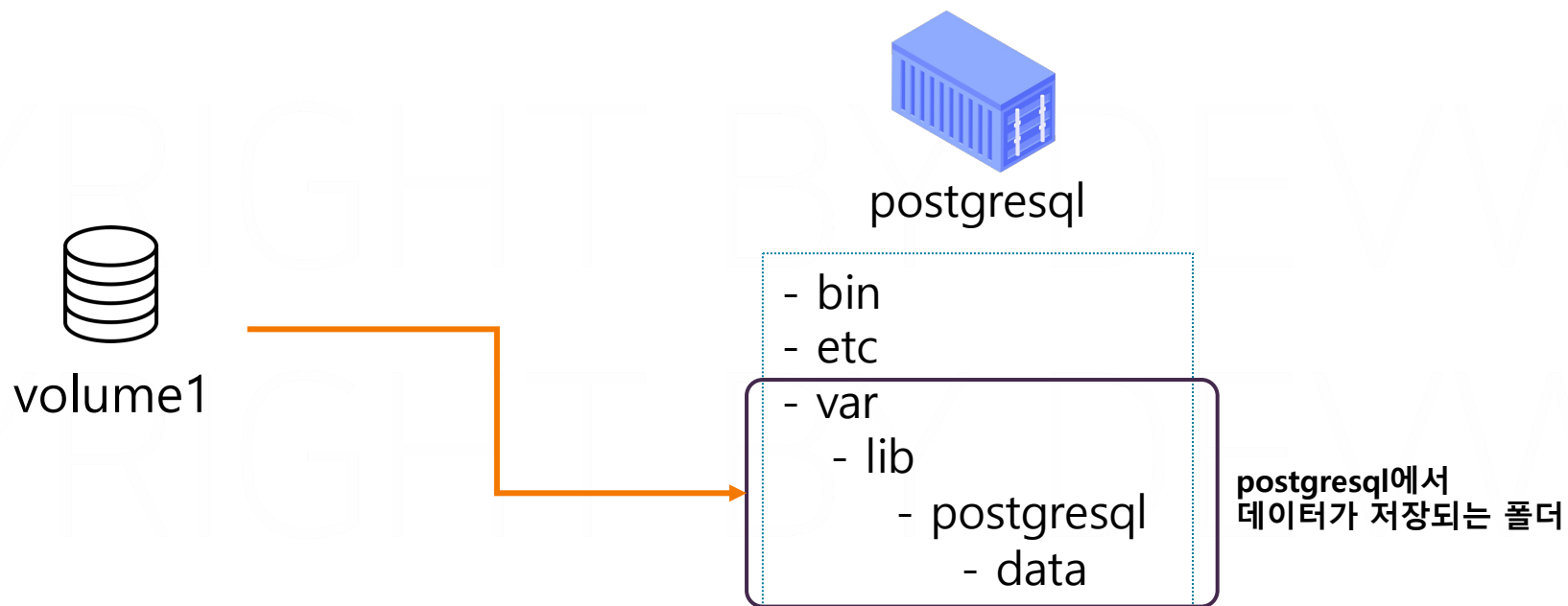


- 컨테이너 실행 시 볼륨을 컨테이너의 내부 경로에 마운트할 수 있습니다. USB를 꽂는 것과 유사합니다.

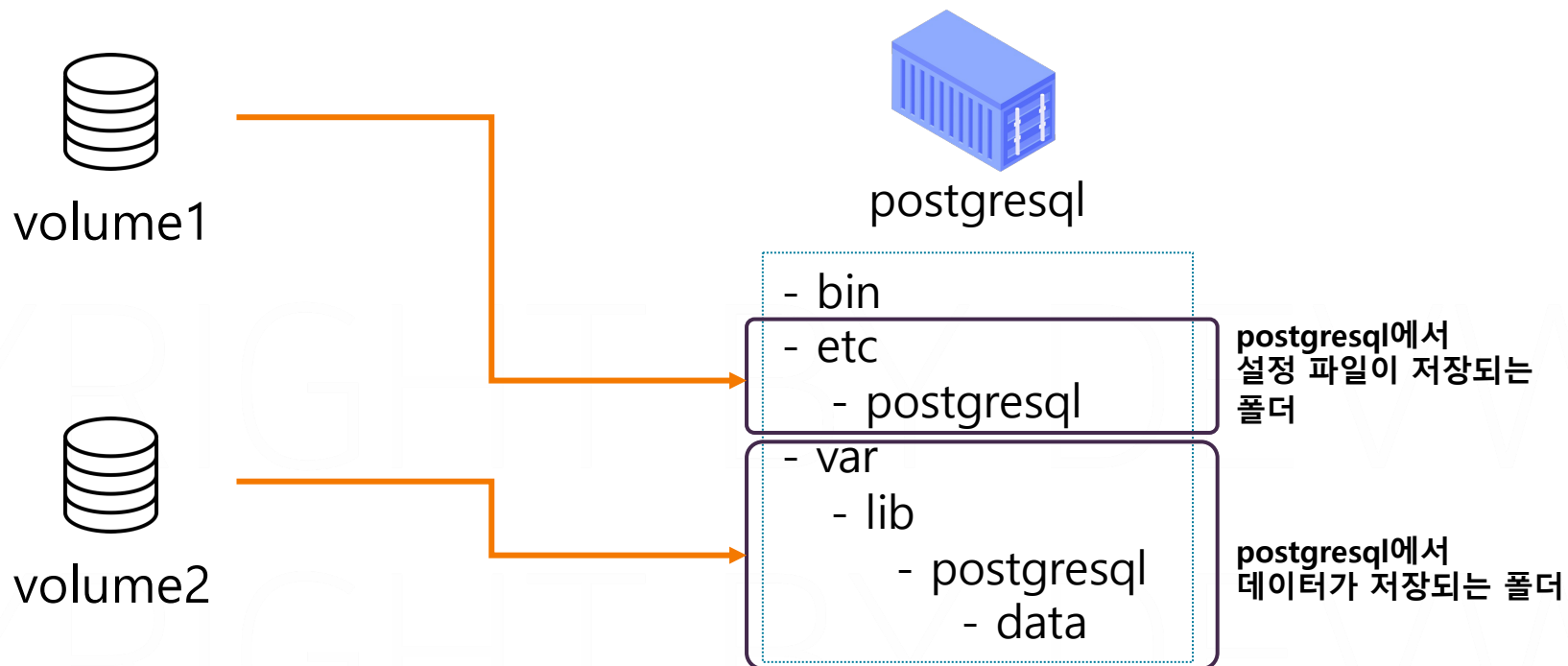
docker run -v volume1:/var/lib/postgresql/data
{도커의 볼륨명} {컨테이너의 내부 경로}



- 컨테이너가 삭제되도 볼륨은 남아 있습니다.
- 컨테이너가 실행 시 다시 마운트 할 수 있습니다.

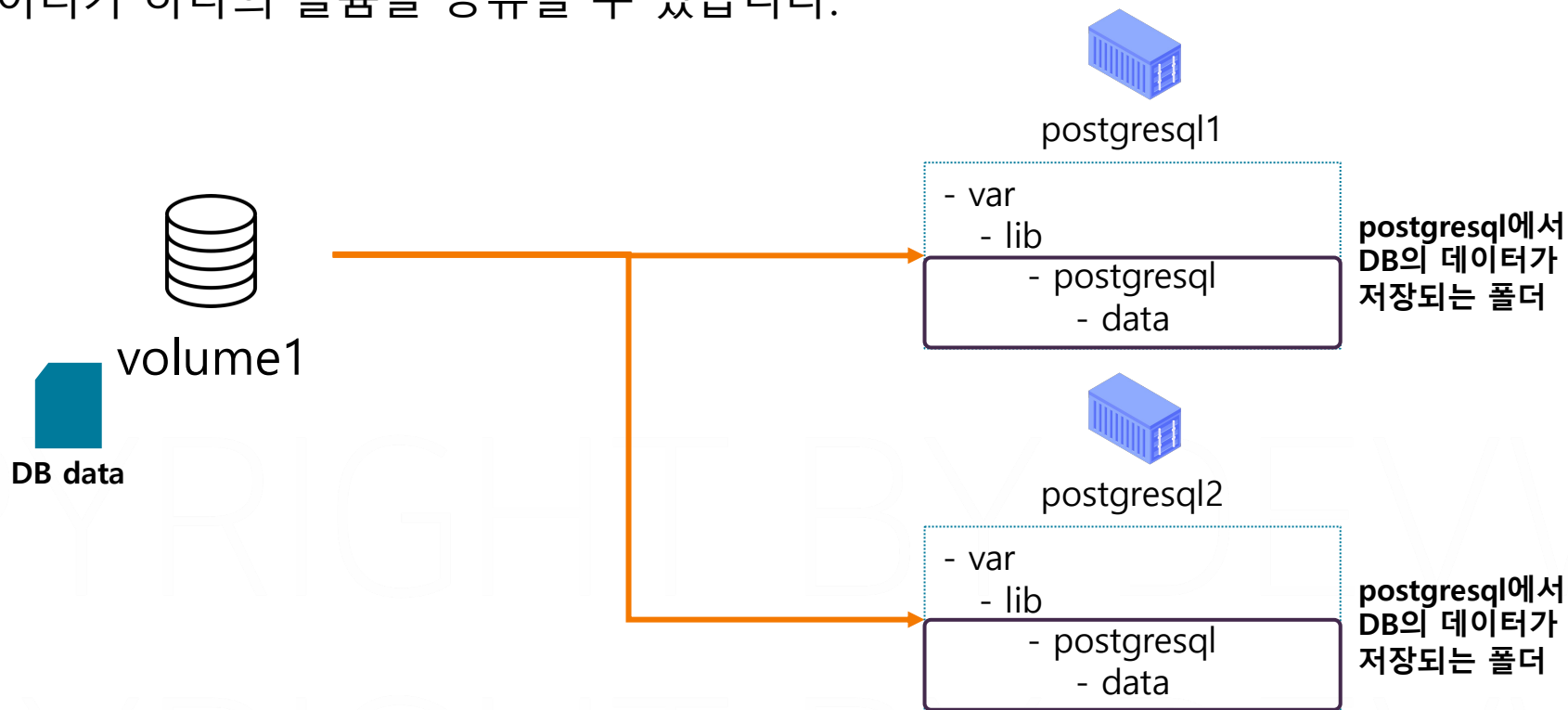


- 하나의 컨테이너가 여러 개의 볼륨을 사용할 수 있습니다.



```
docker run -v volume1:/etc/postgresql -v volume2:/var/lib/postgresql/data
```

- 여러 개의 컨테이너가 하나의 볼륨을 공유할 수 있습니다.



하나의 DB 데이터를 모든 postgresql 컨테이너가 공유하도록 설계

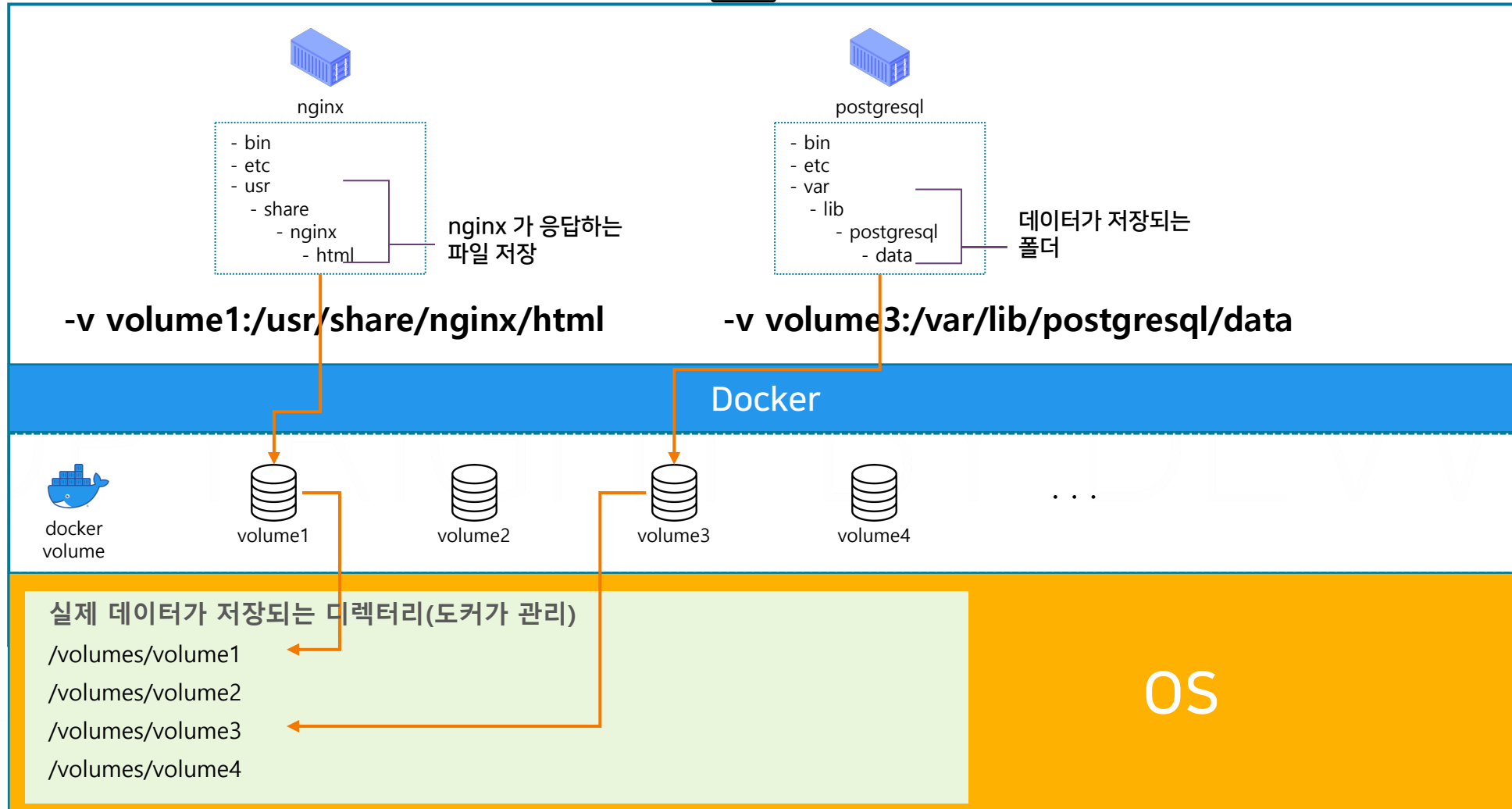
```
docker run -v volume1:/var/lib/postgresql/data --name postgresql1
```

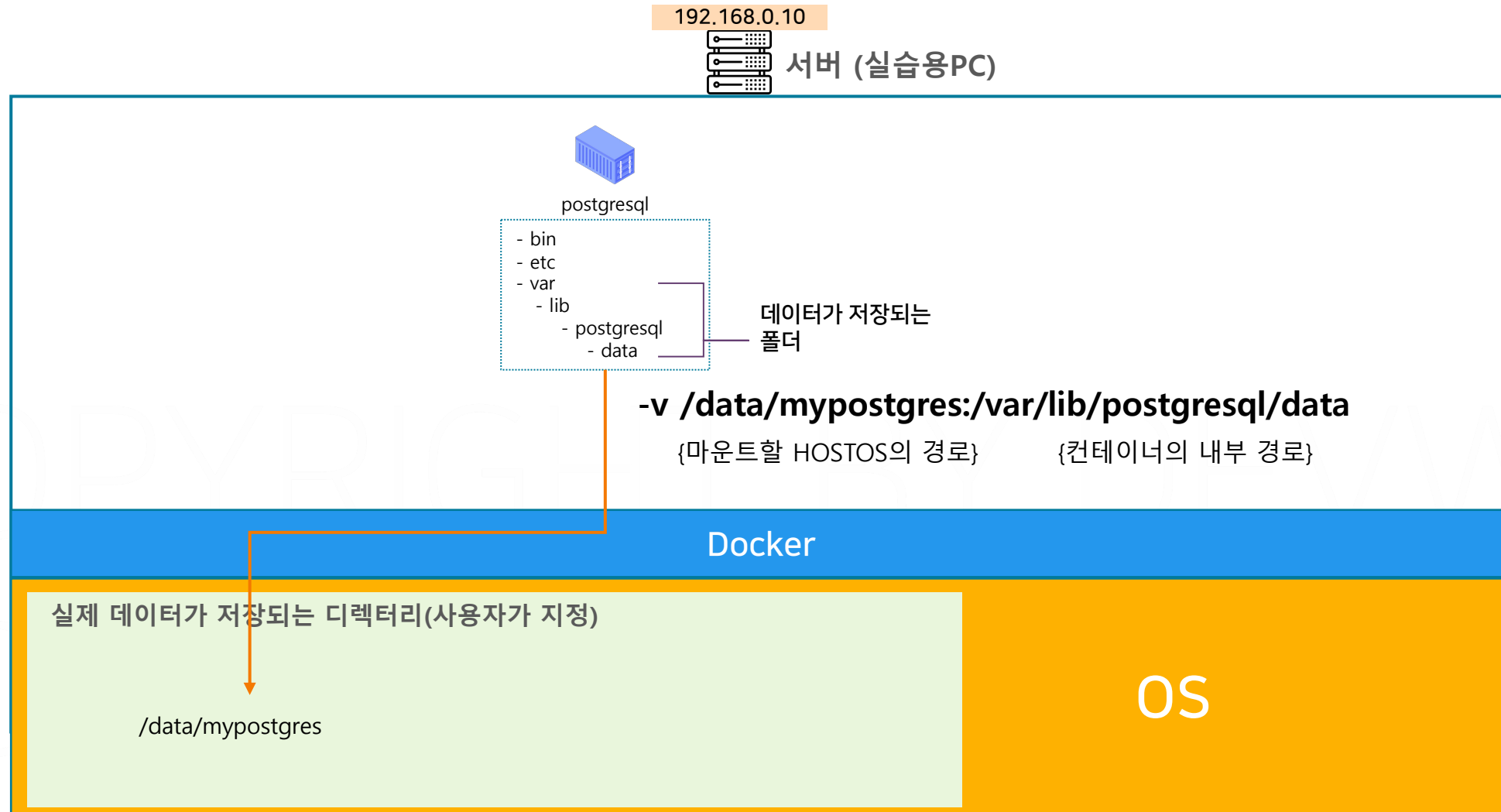
```
docker run -v volume1:/var/lib/postgresql/data --name postgresql2
```

192.168.0.10



서버 (실습용PC)





docker volume ls

볼륨 리스트 조회

docker volume inspect 볼륨명

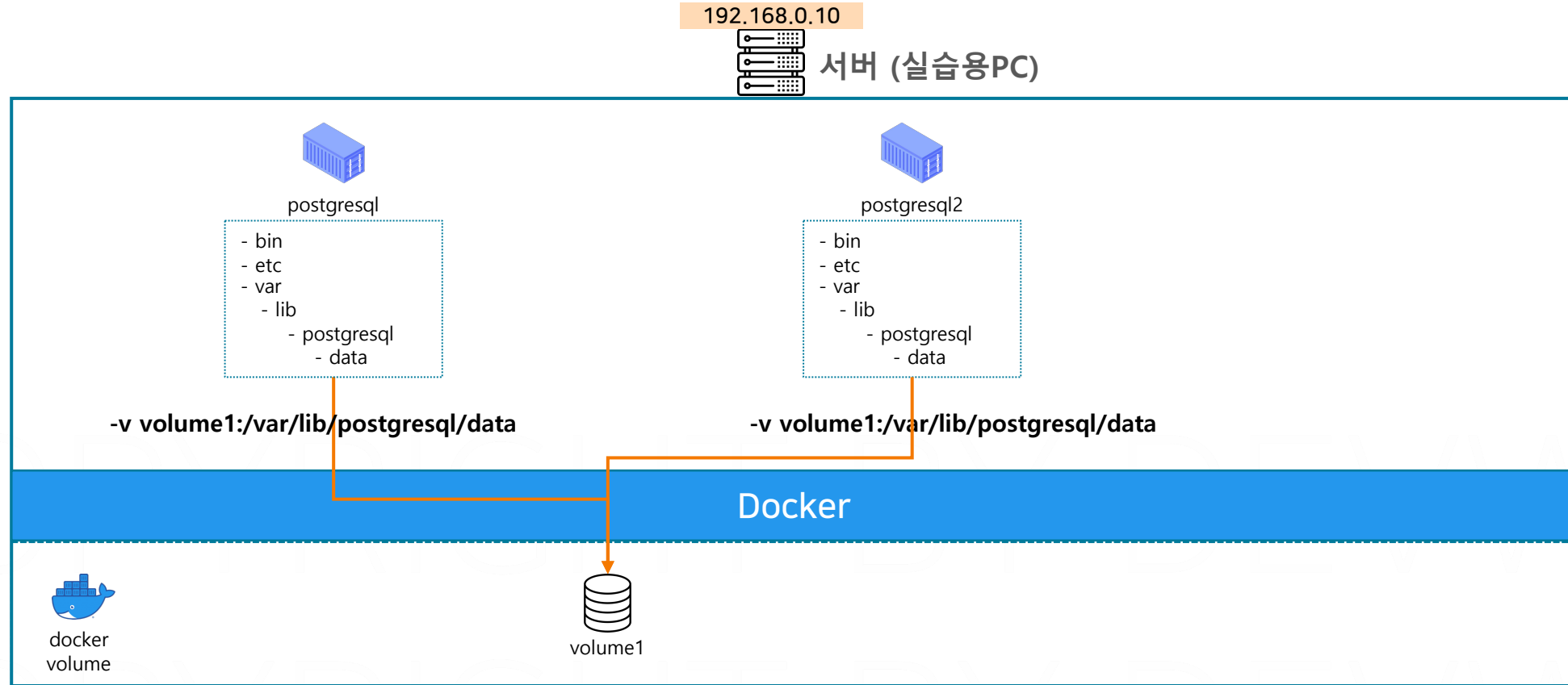
볼륨 상세 정보 조회

docker volume create 볼륨명

볼륨 생성

docker volume rm 볼륨명

볼륨 삭제



- 실제 데이터베이스의 데이터가 **volume1**에 저장
- 컨테이너가 삭제되어도 데이터 영역은 남아 있음
- 새로운 컨테이너 생성 시 기존 데이터 영역 재활용 가능

Postgresql 볼륨 실습

1. 볼륨 생성

```
docker volume create mydata
```

2. 볼륨 리스트 조회 및 상세 정보 조회

```
docker volume ls
```

```
docker volume inspect mydata
```

3. Postgres 컨테이너 생성 및 볼륨 연결, 컨테이너 상세 정보 조회

```
docker run -d --name my-postgres -e POSTGRES_PASSWORD=password -v mydata:/var/lib/postgresql/data postgres:13
```

```
docker container inspect my-postgres
```

4. 컨테이너에 DB 생성(mydb) 명령어 실행

```
docker exec -it my-postgres psql -U postgres -c "CREATE DATABASE mydb;"
```

5. 기존 컨테이너 제거 및 볼륨 상태 확인

```
docker rm -f my-postgres
```

```
docker volume ls
```

6. 새로운 Postgres 컨테이너 생성 및 기존 mydata 볼륨 연결

```
docker run -d --name my-postgres-2 -v mydata:/var/lib/postgresql/data -e POSTGRES_PASSWORD=password postgres:13
```

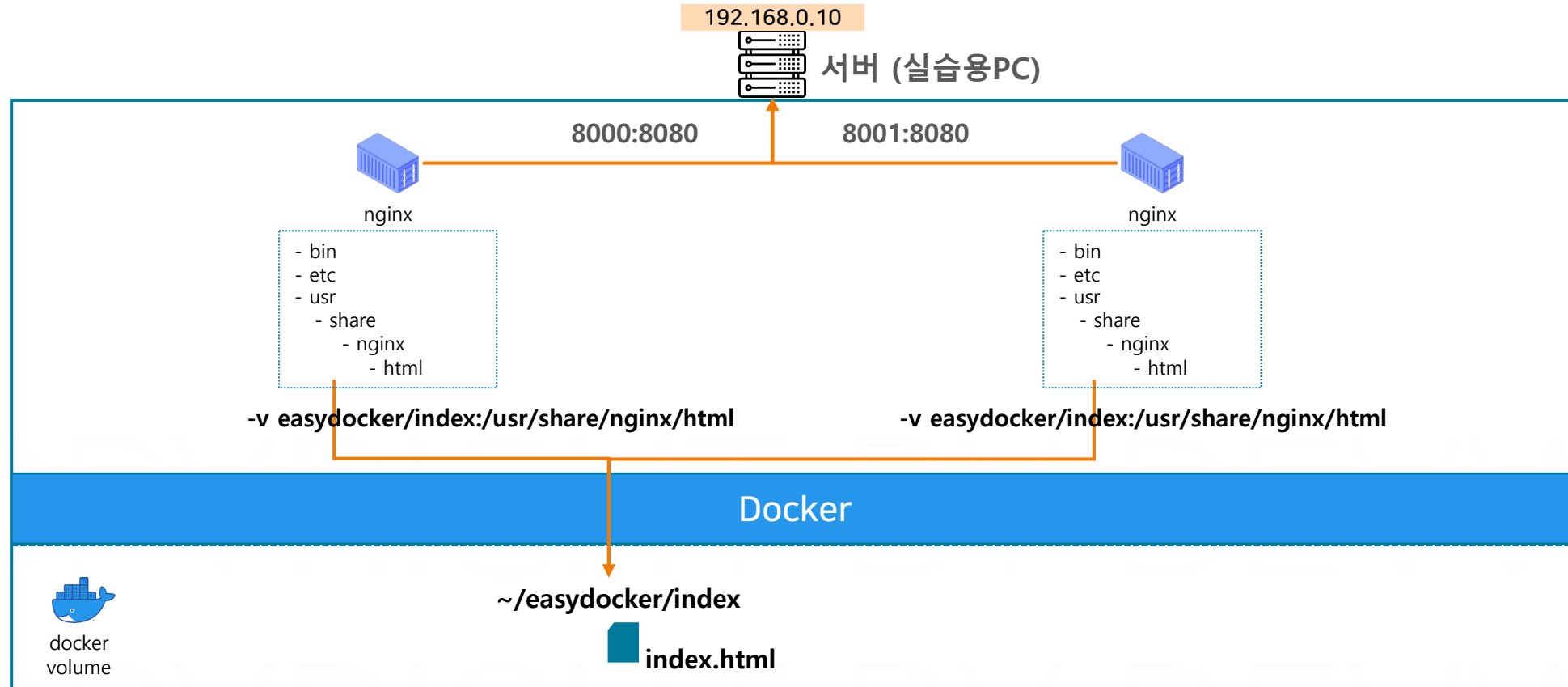
7. mydb가 그대로 유지되어있는지 확인

```
docker exec -it my-postgres-2 psql -U postgres -c "\list"
```

8. 생성한 컨테이너 및 볼륨 삭제(사용중인 볼륨은 삭제 불가)

```
docker rm -f my-postgres-2
```

```
docker volume rm mydata
```

- 호스트OS의 디렉터리를 2개의 컨테이너가 마운트하여 공유
- nginxA에서 변경한 파일이 nginxB도 변경됨
- 바인드마운트는 별도로 볼륨이 만들어지지 않음

Nginx 데이터 공유

1. 폴더 생성 및 Nginx 컨테이너 생성 및 볼륨 연결

```
mkdir index && cd index
```

```
docker run -d -p 8000:80 --name my-nginx-a -v /Users/hyeonwoo/easydocker/index:/usr/share/nginx/html nginx
```

```
docker run -d -p 8001:80 --name my-nginx-b -v /Users/hyeonwoo/easydocker/index:/usr/share/nginx/html nginx
```

2. 볼륨 리스트 조회 (조회되지 않아야 정상)

```
docker volume ls
```

3. localhost:8000, localhost:8001 접속 시도(에러)

4. 생성한 index 경로에 Hello Volume! 내용의 index.html 파일 생성

5. localhost:8000, localhost:8001 접속 하여 내용 확인

6. index.html 파일의 내용을 Bye Volume! 으로 수정

7. 생성한 컨테이너 삭제

```
docker rm -f my-nginx-a my-nginx-b
```